BELGIUM CAMPUS ITVERSITY

SEN381

BUGBUSTERS

CIVICCONNECT PROJECT TEAM

ASSIGNMENT 3(A3)

Research for Quality, Security & Production Readiness Decisions

TEAM MEMBERS

CHRISTIAN JANSE VAN RENSBURG (601840)
MARCO REINERS (578056)
WALDO BLOM (578068)

PROJECT DOCUMENT DETAILS

COURSE	Software Engineering 381 (SEN381)
PROJECT	Assignment 3
SUBMISSION DATE	[28 September 2026]

Table of Contents

Quality Strategy & Risk-Based Verification (Question 1) 1
1.1 Quality Assurance, Quality Control, Verification and Validation in a Quality Strategy 1
1.2 Risk-Based Verification 1
1.3 Complementary Forms of Verification Evidence 1
1.4 Automation, Quality Gates, and the “Green Pipeline” Fallacy 2
1.5 Visual examples of software risks / quality concerns 2
Security Engineering & Threat-Based Controls (Question 2) 3
Threat Modelling and the Selected Approach 3
Threat-to-Control Traceability 3
Relationship Between Secure Design, Secure Coding and Security Checks 5
Limitations of Automated Security Tools and AI-Generated Recommendations 5
Critical Question 5
Production Readiness, Deployment & Operational Evidence (Question 3) 6
1. Environment strategy 6
2. Secrets and production configuration 7
3. Release and deployment 8
4. Failure and recovery 8
5. Operations and observability 9
6. Operational quality 10
Production-readiness Evidence matrix 12
Research-to-M3 Engineering Map (Question 4) 13
References 15
AI usage register 18

Quality Strategy & Risk-Based Verification (Question 1)

1.1 Quality Assurance, Quality Control, Verification and Validation in a Quality Strategy

A quality strategy requires an understanding of four concepts namely, Quality Assurance (QA), Quality Control (QC), Verification and Validation (V&V).

- Quality Assurance is a proactive process-orientated activity. QA focuses on that defects doesn't happen by defining the standards, architectures and processes that an engineer must follow (Sommerville, 2015).
- Quality Control is a reactive product-orientated activity. It involves tests and reviews to detect the defects in software that has already been built (Sommerville, 2015).
- Verification addresses the technical question, "are we building the product right?" Verification evaluates whether the software meets its technical and architectural requirements (Sommerville, 2015).
- Validation answers the business question, "Are we building the right product?" It evaluates whether the software actually satisfies the stakeholders needs and expectations in the real world (Sommerville, 2015).

In a quality strategy, verification effort must not be treated as a final phase. QA lays out how the team will build the software; QC continuously verifies the code against technical specification through automated pipelines; validation ensures that the product holds business value

1.2 Risk-Based Verification

Testing every component of a system equally is financially and technically unfeasible. Risk-based verification solves this by allocating testing effort based on the product-risk, which is calculated by using the likelihood of a failure happening and the impact if it does (Crispin and Gregory, 2009).

Instead of trying to test everything equally, the critical components such as authentication services should receive deeper verification such as automated integration, performance and security testing. On the other hand, low-impact components such as about pages or static tests only requires lightweight or basic unit tests. This ensures that engineering effort is spent mitigating failures that would cause the most business or technical damage.

1.3 Complementary Forms of Verification Evidence

To secure a web-based information system, team must combine different layers of verification, as no single test type can catch every defect.

1. Unit verification (Automated unit tests)
   - It is effective at exposing logical errors, edge-case failures and algorithmic defects in isolated functions, classes or methods.
   - Limitations: It cannot prove that components work together. A unit test will pass if a function works in isolation, even if it is entirely incompatible with the database or the external API it needs to talk to (Fowler, 2012).
2. API / Integration verification
   - It exposes contract failures incorrect data formatting and communication breakdowns between boundaries for example, between the backend service and the database, or between the backend and a third-party email API (Richards and Ford, 2020).
   - Limitation: It does not prove that the UI is functional, and it does not guarantee that the end-to-end business workflow makes sense to a user.

3. System / End-to-End (E2E) verification
    - It exposes complete system flow failures and broken end-user journeys by testing the application from the UI down to the database, exactly as a user would experience it.
    - Limitations: E2E tests are notoriously slow, high brittle, which is easily broken by small UI changes, and make it difficult to pinpoint the exact root cause of failure when they break (Humble and Farley, 2010).

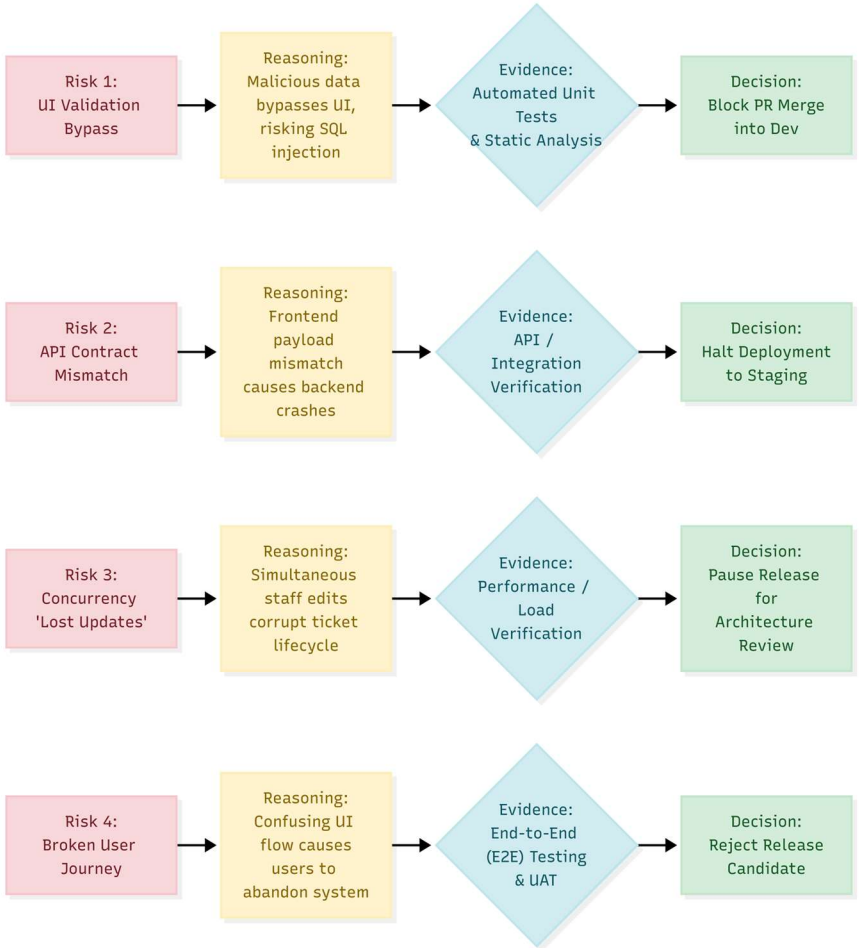
1.4 Automation, Quality Gates, and the "Green Pipeline" Fallacy

Automation and quality gates make verification repeatable, objective and scalable. By configuring a Continuous Integration (CI) pipeline to act as a quality gate, a team can automatically block code from merging if it fails compilation, unit test or analysis checks

Critical Question Analysis: Why is 'all automated tests passed' insufficient evidence by itself to conclude that software product is high quality or ready for release?

A "green pipe" only proves that the specific assertions written by the developers have executed successfully. It is limited by the developer's vision. It cannot prove the unknown bugs or edge cases that were not scripted. Automated test (verification) cannot evaluate usability, accessibility, or whether the feature genuinely the feature solves the client's business problem (validation). A system can have 100% passing automated tests while still being entirely unusable or functionally incorrect for the end-user (Fowler, 2012).

1.5 Visual examples of software risks / quality concerns



Security Engineering & Threat-Based Controls (Question 2)

Threat Modelling and the Selected Approach

Threat modelling is a process in which a system is examined from a security point of view (Tran, et al., 2026). Possible treats are identified and ways to mitigate those threats are decided. It helps developers to think about how a system could be misused or attacked, rather than just focusing on the intended functionality of a software system (OWASP, 2026c).

Threat modelling should be done initially during the planning phase of the Software Development Life Cycle (SDLC) and continue as the system evolve. This allows security concerns to be identified before they are built into the design or implementation of the system. It also allows security to form part of the development process instead of only being considered near the end through activities such as penetration testing (OWASP, 2026c).

A recognised threat-modelling approach suitable for software engineering is STRIDE. STRIDE helps developers systematically consider possible threats relating to spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privilege. These categories are used as prompts to help identify how threats could apply to the specific system being analysed, rather than simply being used as a checklist of security terms (OWASP, 2026c). Microsoft's Threat Modelling Tool also uses STRIDE as part of analysing software designs for possible security issues and identifying suitable mitigations (Microsoft, 2022).

The main value of STRIDE is that it supports decisions about what risks need to be addressed and how they should be handled. Once a threat has been identified, its likelihood and impact can be considered to help determine its priority. The development team can then decide whether the threat should be mitigated, eliminated, transferred or accepted. Where mitigation is required, the response can be turned into an actionable security requirement that can later be implemented and verified (OWASP, 2026c). This creates a clear process from understanding the system, identifying threats, assessing their importance, selecting a response and then verifying that the chosen controls work as intended.

Threat-to-Control Traceability

Security concerns should lead to clear and traceable engineering responses rather than being treated as isolated risks. For each concern, a plausible threat or misuse should first be identified and then linked to a specific security requirement or control objective. This requirement can then be translated into a practical engineering control, with suitable verification evidence used to show whether the control has been implemented and is working as intended.

This traceability points out the reasoning behind each security decision and makes it easier to understand why a particular control is needed. It also helps to avoid that a control or successful test is treated as complete proof of security, because some residual risk can remain even after the control has been implemented and verified.

The table below applies this process to authentication, authorisation/access control, input/data handling and dependency vulnerabilities. It shows how each concern leads to different requirements, controls and forms of verification evidence. It also identifies the limitations or risks that may remain.

Security concern	Threat / misuse	Security requirement or control objective	Possible engineering control	Verification evidence	Residual risk / limitation
Authentication	An attacker attempts to gain access to another user's account by guessing passwords, using stolen credentials or repeatedly attempting logins.	The system should verify that users are who they claim to be and reduce the risk of unauthorised access through credential-based attacks.	Use strong password controls, login throttling or other protections against automated login attempts and, where appropriate, multi-factor authentication. These controls help reduce brute-force, credential-stuffing and password-spraying attacks (OWASP, 2026a).	Test valid and invalid login attempts, repeated failed-login attempts, throttling or lockout behaviour and MFA flows where applicable. The evidence should show that unauthorised authentication attempts are rejected (Souppaya, et al., 2022).	Authentication controls do not remove all account-compromise risk. Valid credentials or session tokens may still be stolen through phishing, malware or a compromised device. Successful authentication also does not prove that the user is authorised to access every resource (OWASP, 2026a; OWASP, 2026b).
Authorisation / Access Control	An authenticated user attempts to access data, functions or resources that they should not be allowed to use, for example by changing an identifier, URL or API request (Mohamed, et al., 2022).	Access should only be granted when the user has the required permission for the requested action or resource. Access control protects assets and information against unauthorised access and modification (Mohamed et al., 2022).	Apply server-side access-control checks, follow least privilege, deny access by default and validate permission on every protected request (OWASP, 2026b).	Use positive and negative access-control tests with different users and roles. API or integration tests should attempt protected actions using users with insufficient permissions, supported by code review of the access-control logic (OWASP, 2026b).	A missed permission check, incorrect rule or unexpected combination of users, roles and resources may still create an access-control weakness. Correct authentication therefore does not automatically prove correct authorisation (OWASP, 2026b).
Input / Data Handling	An attacker submits malformed or malicious data through forms, API requests or other inputs in an attempt to manipulate application behaviour or trigger attacks such as injection (Blancaflor, et al., 2024).	Data from untrusted sources should be validated before it is processed so that malformed or malicious input does not reach downstream components or cause unintended behaviour. Input validation has been shown to reduce exposure to injection-related attacks in web applications (Blancaflor et al., 2024).	Perform server-side input validation as early as possible, using checks such as allowed values, expected format, data type, length and range. Input validation can help reduce injection-related risk, although it should not be treated as the only defence against attacks such as SQL injection or XSS (Blancaflor, et al., 2024; OWASP, 2025a).	Use unit and API tests with valid, invalid and deliberately malformed input. Static analysis, targeted security testing and code review can provide additional evidence that validation is consistently applied (OWASP, 2025a).	Input validation alone cannot prove that injection or other data-handling attacks are impossible. Other controls are still needed, and unexpected inputs or application-specific data flows may still be missed (OWASP, 2025a).
Dependency Vulnerabilities	A third-party or transitive dependency contains a known vulnerability that could be exploited through the application. Dependencies can allow vulnerabilities to propagate through a software supply chain beyond the directly affected component (Shen et al., 2025).	Third-party components should be identified, monitored and assessed so that known vulnerabilities can be addressed before they create unacceptable risk. This is important because software supply-chain vulnerabilities can originate and propagate through dependency relationships (Shen et al., 2025).	Maintain dependency information, monitor components for known vulnerabilities, and update or patch vulnerable dependencies when a suitable fix is available. Automated dependency analysis should be introduced early rather than only near the end of development (OWASP, 2025b).	Use dependency or software-composition scan reports to identify known vulnerable components. After an update or patch, rerun relevant unit, integration, functional or security tests to confirm that the change addresses the issue without breaking expected behaviour (OWASP, 2025b; Souppaya, et al., 2022).	Dependency scanners mainly identify vulnerabilities that are already known and represented in their data sources. Transitive dependencies and repair or compatibility constraints can also make remediation more difficult, while the presence of a vulnerability does not automatically prove that it is exploitable in the specific application (OWASP, 2025b; Shen, et al., 2025).

Relationship Between Secure Design, Secure Coding and Security Checks

Secure design, secure coding and security checks support different parts of software security and should therefore work together. Secure design supports system requirements and architecture, where threats, trust relationships and required protection are identified before implementation (Rouland, et al., 2021). Secure coding then turns those design decisions into working software. Security and dependency checks provide evidence that the implementation still meets the intended security requirements (Al-Qadda, et al., 2026).

It is important to note that these activities cannot replace one another. They all address different stages of the same security process (Souppaya, et al., 2022). Secure design determines what protection is needed. Secure coding implements those decisions. Thereafter security and dependency checks provide evidence that the implementation still meets the intended requirements. A weakness in any one of these areas can weaken the effectiveness of the others (Al-Qadda, et al., 2026). Because of this it is crucial that they need to work together throughout development (Souppaya, et al., 2022).

Limitations of Automated Security Tools and AI-Generated Recommendations

Automated security tools and AI-generated recommendations can support security engineering, but they should not be treated as complete proof that a system is secure.

- **Limited coverage and context:** Automated tools can miss weaknesses because they do not always understand the full design, business rules or security requirements of the system. They can produce false positives, where safe code is incorrectly flagged as a problem, creating unnecessary work for developers. A more severe problem is that automated tools sometimes produce false negatives, where real vulnerabilities are present (Al-Qadda, et al., 2026). This means that a clean scan does not necessarily mean that no security weaknesses are present.
- **AI recommendations still require human verification:** AI can assist with activities such as code generation, vulnerability detection and threat modelling, but its recommendations may still be incomplete, insecure or unsuitable for the specific system being developed. As mentioned, AI sometimes overlook important context or encourage over-reliance on generated recommendations (Al-Qadda, et al., 2026). Security-aware prompting alone does not guarantee fewer vulnerabilities. Therefore AI-generated recommendations still require human review and independent verification.

Automated tools and AI can support secure design, coding and verification but cannot replace human insight. NIST recognises that automation does improve the accuracy and repeatability of security practices but still requires defined security criteria and manual practices where automated methods are not sufficient (Souppaya, et al., 2022).

Critical Question

A security scanner reporting no high-severity findings does not prove that the system is secure. Automated security tools have limitations in both coverage and accuracy and may still produce false negatives where real vulnerabilities are not detected (Dalaq, et al., 2025). Static analysis tools struggle with complex vulnerabilities and application-specific context (Mweu & Ndia, 2025). This implies that important weaknesses may remain undiscovered even when a scan appears clean.

Important security claims may therefore remain unproven. This includes whether access-control rules are correct, whether business logic can be misused, whether sensitive data is properly protected, or whether all security requirements have been implemented correctly. Automated checks should therefore be treated as one form of security evidence rather than complete proof of security (Souppaya, et al., 2022). Security assurance should still be supported by other activities such as threat modelling, code review, targeted security testing and human review. These activities identify

weaknesses that automated tools may miss or don't not fully understand (Al-Qadda, et al., 2026). Therefore, a clean scan should be seen as useful evidence, but not as confirmation that the entire system is secure.

Production Readiness, Deployment & Operational Evidence (Question 3)

An important decision in software development is deciding whether a product is ready for release at any given point in the development lifecycle. As Satapathy (2008) states in his evaluation of software release readiness metrics, this decision must be made at the end of each release sprint or iteration. However, such decisions are frequently made informally or through a simplistic approach that relies on hidden metrics rather than comprehensive and systematic evaluation (Satapathy, 2008).

Discussed below is what must be considered before a software product can be described as production ready:

1. Environment strategy

Production readiness begins with the separation across multiple distinct environments, each with its own purpose (Microsoft, 2023). A multi environment approach allows teams build, test, and release code with a greater speed and frequency (Microsoft, 2023). Examples of these environments include:

Development environment

The development environment is where changes to software are made. This is where developers write, run and test code locally or in a shared sandbox environment (Microsoft, 2023). The development environment is designed for rapid iteration and debugging, allowing developers to experiment and build features without impacting others (Microsoft, 2023).

Test environment

The test environment allows for either human testers or automated tests to test new code in a controlled, reproducible setting (Microsoft, 2023). Before code enters test environments, developers must accept new code and configurations through unit testing in the development environment (Microsoft, 2023). Test environments are important because they allow developers to test changes before deploying it to production, which reduces risk related to bugs within production (Microsoft, 2023).

Staging environment

The staging environment is the last gap between testing and production (Microsoft, 2023). Staging is where final testing occurs directly before to deploying to production (Microsoft, 2023). Each staging environment should mirror an actual production environment as accurately as possible, including hardware, servers, databases, and network configurations (Microsoft, 2023). This environment provides the last opportunity in order to identify and resolve issues before the software is released to the end users (Microsoft, 2023).

Production Environment

The production environment, also commonly referred to as "live," is the environment that end users directly interact with (Microsoft, 2023). Production operates under strict controls: restricted deployment permissions, fixed infrastructure patterns, and detailed monitoring (Microsoft, 2023). Changes to production are implemented through carefully controlled processes with designated reviews and approvals (Microsoft, 2023).

Why uncontrolled environment differences create deployment risk

If there are differences in the configuration of these environments, "configuration drift" can happen, which can result in data loss, slower deployments, and failures (Microsoft, 2023). Teams can speed up deployments, improve environment consistency, and reduce configuration drift by adopting Infrastructure as Code (IaC) (Microsoft, 2023). Additionally, automated checks on test results should be used in order to control the transition of code from development to production, thereby preventing failing tests from automatically deploying changes to the next environment (Microsoft, 2023).

2. Secrets and production configuration

Secrets are being used everywhere especially within DevOps (OWASP, 2026d). It includes things such as API keys, database credentials Identify and Access Management (IAM) permissions, SSH keys, certificates etc (OWASP, 2026d). Secrets should never be stored within the source code as plain code (OWASP, 2026d). Thus, originations need a way to centralize the storage, provisioning, auditing, rotation and management of these secrets in order to control access to these secrets and prevent them from leaking.

How to separate secrets from the source code:

1. Make use of a secrets management system. This includes cloud provider facilities such as AWS Secrets Manager, Azure Key Vault, and Google Secret Manager, or third-party tools such as Hashicorp Vault, Conjur, or Keeper (OWASP, 2026d). The goal is to centralise and standardise the secrets management solution in order for the secrets to remain maintainable and useable (OWASP, 2026d).
2. CI/CD tools that allow for secret storage . This includes things such as GitHub secrets, GitLab repository secrets, ENV Vars/Var in Microsoft Azure DevOps, Kubernetes Secrets, etc (OWASP, 2026d). These secrets are often configurable and viewable by people who have the authorization to do so (e.g., a maintainer in GitHub, a project owner in GitLab, an admin in Jenkins, etc.) (OWASP, 2026d).
3. Encrypted secrets committed to Git is another alternative, the CI/CD pipeline can use Encryption-as-a-Service from the secrets management system to encrypt a secret, which is then committed to Git and decrypted by the consuming service on deployment (OWASP, 2026d).
4. .env files is also a common option that is used but the mistake that is often made is that this .env file is committed directly to the repository (OWASP, 2026d). This is not safe as the secrets are stored as plain text and are then exposed. .env files are only intended for local development and should never be committed via git (OWASP, 2026d).

Controlling secret configuration

Controlling production configuration of secrets is more than just separating the secrets from the source code. Secrets should be centralized in a dedicated secret management system in order to enable consistent auditing, rotation and access control. Access must be controlled using the Least Privilege principle, so that no engineer has access to every single secret (only the ones that they should have access to.) (OWASP, 2026d). Secret automation should also be used where possible as manual secret maintenance increases human error and risk of secret leakage, this includes things such as automatic secret rotation (OWASP, 2026d). Lastly, it is also recommended that all secret access and pipeline actions should be logged in order to keep a log of who accessed what and when they accessed a specific secret (OWASP, 2026d).

3. Release and deployment

Deployment is the process of moving software from one controlled environment to another, for example from development to testing.

Release is a business decision about making services or features available to end users. According to ISO/IEC 2000 a software release is directly defined as “A collection of one or more new or changed services or service components deployed into the live environment as a result of one or more changes.” (Mathenge, 2020). A company can decide to release features based on an agreed marketing plan or features can be released to different customers based on the company’s product offerings (Mathenge, 2020).

DevOps practices involve frequent release cycles that provide benefits to organizations and their end users in many ways. In order to reduce the possibility of problems, a deployment strategy that best suits the organization's needs must be adopted (Mathenge, 2020).

Traditionally, deployments were carried out using what is referred to as “the big-bang approach” which is where all features are released at once. But the more common practice now is rolling or phased deployments through a gradual release across the environment over a period of time (Mathenge, 2020).

One common technique used in deployment is the “blue-green approach” (Mathenge, 2020). This is where, two identical but separate production environments are used, with the current code running on the ‘blue’ environment, and the new code being deployed on the ‘green’ environment (Mathenge, 2020). Traffic is then switched from blue to green, and the performance of the new service or features is then monitored. In case of a major problem, traffic is switched back to blue (like a rollback), allowing for fixing and remedies without affecting customers significantly (Mathenge, 2020). Other examples include the canary strategy and the rolling strategy (Oracle, 2026).

CI/CD has allowed for faster deployment of software with fewer errors due to the help of automation in integration, testing and deployment, this is achieved by using tools such as Gitlab and Jenkins (Mathenge, 2020). However, there are still some organizations that prefer manual change approvals to trigger deployment in the production environment for reasons such as to meet compliance needs (Mathenge, 2020).

The key difference between deployment and release is the business decision (Mathenge, 2020). Deployment doesn’t necessarily mean users have access to features, as can clearly be seen by the different environments involved. Some companies will release at the same time as deployment to production is taking place, but this is not always the case (Mathenge, 2020).

In order to ensure that deployment is successful, the deployment activities should be followed, which include planning the deployment, verifying service components, verifying target environment, executing the deployment and lastly confirming the deployment by doing acceptance testing to validate customer requirements are met (This is where post review and lessons learnt actives are recorded and carried out) (Mathenge, 2020).

4. Failure and recovery

When adding a new feature to a service there is always the risk of something breaking, even when doing unit and functional tests in order to cover possible places where the code can break (Hilton, 2024). This is why it is important to have a failure and recovery plan.

It consists of the following: detecting when a new release is actually broken, the moving of users (safely) from a bad release to a fixed release and lastly preventing too many users from suffering through a bad release in the first place (known as “canarying”) (Hilton, 2024).

For software releases the two most common approaches are rollback and roll forward (Hilton, 2024). Rollback is when the system is reverted to a previous known good version of the software (one without the current detected bug), thereby undoing the failed release (Hilton, 2024). Roll forward is when a fix or new version is applied on top of the failed release in order to resolve the issue without reverting to an older version (Hilton, 2024) . Important to note is that a roll forward is not normally recommended and the accepted practice is a roll back. (Hilton, 2024) Also, very important when doing failure planning the rollbacks should also be tested to ensure that it actually works.

Another consideration when creating a backup strategy also includes determining the Recovery Time Objective (RTO) and the Recovery Point Objective (RPO) (IBM, 2021). RTO is the maximum acceptable time that a system can be unavailable after a failure before it must be restored to operation (IBM, 2021). RPO is the amount of data you can afford to lose, it influences how frequently you need to back up your data in order to avoid losing more (IBM, 2021). You also need to consider where the actual data for the backup will be stored (IBM, 2021). Options include on-premises secondary storage, offsite physical storage, cloud storage, or a hybrid approach combining multiple locations (IBM, 2021). It is important to note that backups should not be stored on the same server / device as the current production one (IBM, 2021).

5. Operations and observability

Production reediness requires that operators can understand what the system is doing, diagnose failures and respond to incidents. Observability consists of three pillars of data in order to make computing networks easier to visualise and understand, it consists of metrics, logs and traces(IBM, 2025) . Additionality alerts can also be used.

Metrics

Metrics are used to provide quantitative insights into system performance by measuring the different network parameters (IBM, 2025). They help teams to understand what the actual system issues are. Examples include Host metrics (memory, disk and CPU usage), network performance metrics (uptime, latency and throughput), app metrics (response times, requests and error rates) (IBM, 2025). Metrics are often combined in order to provide a summary view that uses dashboards and other visualizations (such as time-series graphs) in order to help developers assess the overall health of the system, analyse data trends and also respond to network problems (IBM, 2025). They also inform decisions about scaling and resource allocation, making metrics useful for capacity planning and load management (IBM, 2025).

Logs

Logs are immutable records of events that occur within the system (IBM, 2025). Meaning they can’t be updated / changed only add to. Log files store detailed information about system behaviour and application process such as: event timestamps, transaction IDs, IP addresses, user IDs, error messages and connection attempts(IBM, 2025). When an error, security breach or compliance issue occurs, logs provide the details needed to trace the root cause and understand actually what went wrong (IBM, 2025). Logs offer insights into system events and issues (IBM, 2025).

Traces

Tracing is used to help site reliability engineers and software engineering teams understand the “where” and “how” of system events and issues (IBM, 2025). Tracing data can include: the duration

of network events and operations, the flow of data packets through the architecture, the order in which requests move between different network services and the root cause of system errors (IBM, 2025). Tracing is especially useful in a microservices architecture, where requests can move through multiple, scattered services before reaching their destination (IBM, 2025). Tracing features in observability tools are necessary for latency analyses, which can help engineers in order to identify problematic components and underperforming services that can create performance bottlenecks for end users (IBM, 2025).

Alerts

Alerts are automated notifications sent to operators when a condition exceeds a threshold, triggering human investigation and response (Ewaschuk and Beyer, 2017). Alerts should indicate what is wrong and what the appropriate action is (Ewaschuk and Beyer, 2017). Excessive alerts (alert fatigue) can cause operators to ignore them and insufficient alerts could miss a real problem. Alerts must be configured for critical failure modes (requests failing, error rate spiking, services becoming unhealthy) (Ewaschuk and Beyer, 2017).

Why collection data without actions interpretation is insufficient

Collecting logs, metrics, traces and alerts without actionable interpretation is insufficient for production readiness (Ewaschuk and Beyer, 2017; IBM, 2025). Examples include:

- Metrics are collected but not preserved. This leads operators not being able access historical data to understand trends
- Logs are generated but not indexed. This causes that the log file must be manually examined as it can be searched efficiently.
- Alerts are configured but not detailed or specific enough. This leads operators not being provided with an explanation of what each alert means and what action to take.
- Monitoring dashboards exist but are so cluttered with data that critical signal is lost in noise.

Achieving a successful end product includes choosing to alert on symptoms or possible real problems, adapting the targets to goals that are actually achievable, and making sure that the monitoring implemented supports fast diagnosis (Ewaschuk and Beyer, 2017).

6. Operational quality

According to Cox et al. (2025) operation quality depends on how well a system addresses reliability, performance, scalability, cost and supportability. These attributes influence whether a system can be operated safely, recovered after a failure occurs, and maintained over time.

Reliability

Reliability is the system's ability to consistently perform the intended functions and ability to recover from failures (Cox et al., 2025). Cox et al. (2025) recommends redundancy, fault-tolerant design, monitoring, and automated recovery. Reliability is important for production readiness as downtime can lead to loss of revenue, reduced productivity, and reputational damage. Data integrity is also a part of this including backups, replication, and redundancy that are also needed in order to prevent data loss or corruption (Cox et al., 2025). This could require upfront investment. A system without a defined reliability strategy, resilience mechanisms, or clear ownership is not production-ready system (Cox et al., 2025).

Performance

Performance optimisation is something that needs to be done continuously. Cox et al. (2025) describes defining performance requirements for each layer, designing and deploying, monitoring and

analysing, and then optimising at a later stage (Cox et al., 2025). Teams should use elastic and scalable design patterns, monitor with logs, traces, metrics, and alerts, and adjust the workloads as it changes (Cox et al., 2025). Performance affects the end user satisfaction and also the revenue, and cost of the company producing the product (Cox et al., 2025). For production readiness, this means performance requirements must be specific and clear, the architecture must be able to scale elastically, and continuous monitoring must be in place in order to detect decline (Cox et al., 2025).

Scalability

Scalability is also an important aspect and must be designed into the system. Cox et al. (2025) recommends elastic and scalable design patterns, load testing to simulate realistic traffic, and stress testing in order to find bottlenecks. Autoscaling can also help meet peak demand while reducing cost during low usage by scaling on metrics such as CPU and memory (Cox et al., 2025). A production-ready system needs evidence from load and stress testing, plus a defined scaling strategy that includes autoscaling policies or manual procedures. Scalability decisions must be informed by performance testing, not by assumptions (Cox et al., 2025).

Cost

In the cloud, you pay for the resources that you consume, it works on a pay-as-you go model. Cox et al. (2025) recommends balancing spending with business value, by planning for only what is needed. Performance optimisation can also reduce cost, for example, autoscaling removes unused resources during low load (Cox et al., 2025). Cost is a production-readiness concern because uncontrolled cloud spending can make a product financially unsustainable and very expensive (Cox et al., 2025). A production-ready system should have a defined cost model, budget alerts, and a strategy for continuous cost optimisation (Cox et al., 2025).

Supportability

Once a product is finally shipped and live, issues are inevitable (Cox et al., 2025). What matters is how these issues are dealt with. This includes how quickly problems are detected, owned, and resolved across different teams (Cox et al., 2025). A system is not production-ready if there is no operational support model, no clear ownership, no observability, and no incident management process (Cox et al., 2025). Supportability means the team can operate, monitor, and maintain the system after the initial release. Cox et al. (2025) refers to this as the “operational excellence pillar”

Production-readiness Evidence matrix

Production-readiness concern	Risk if ignored	Evidence before release	Evidence after release
Environment Parity and Configuration Management	Uncontrolled differences between development, test, and production environments can cause deployment failures or unexpected behaviour.	Automated validation confirming staging mirrors production (hardware, servers, databases, network etc); configuration drift audit. Automated checks preventing failed tests from becoming part of the code base.	Continuous monitoring for configuration drift. Automated alerts. Regular audits confirming production maintains standards.
Secrets Management	Hardcoded secrets, API keys, and database credentials in source code can expose sensitive information and cost the company money.	Implement a secrets management system. Go through logs in order to confirm no hardcoded secrets in the repository. Access control demonstrating least-privilege. .env files excluded from Git.	Access audit trail (who, what, when); regular scans confirming externalized secrets. Alerting on unauthorized access.
Rollback and Recovery Readiness	Without tested rollback and database compatibility plans, a failed release cannot be reversed safely.	Tested rollback procedures executed at least once. Document RTO and RPO. A successful disaster backup, recovery and restoration test.	Regular backup integrity verification through restore testing. Rollback procedures tested periodically. Post-incident reviews documenting rollback effectiveness.
Backup and Data Recovery	Data loss or corruption without tested backup and restore procedures.	Backup strategy documented (location geographically separate from production). Restore procedures documented and tested. RPO calculated.	Continuous backup execution logs. Compliance audits confirming procedures maintained.
Observability: Metrics, Logs, and Traces	Operators cannot diagnose failures or understand system behaviour in production.	Metrics collection strategy defined (host, network, application). Centralized logging with indexing established. Dashboards displaying system health. Historical metrics retention configured.	Dashboard actively monitored. Logs searchable for incident diagnosis. Traces guide latency analysis.
Monitoring and Alerting Configuration	Alerts that are too noisy cause alert fatigue and alerts that are too sparse miss real problems.	Alert thresholds defined for critical failures (request failures, error spikes, unhealthy services). Tested in order to confirm correct triggers. Messaging is specific and actionable.	Alert execution logs. Response-time metrics showing operators diagnose effectively. Periodic threshold tuning.

Research-to-M3 Engineering Map (Question 4)

The map below transfers the strongest findings from Questions 1-3 into candidate evidence and approaches for Milestone 3. It does not select a CivicConnect test strategy, security control, deployment platform or monitoring solution. Final M3 decisions should be made only after the updated PED v2 architecture and design baseline is available and can be evaluated against the implemented system, current risks and project constraints

Research finding	Engineering concern it addresses	Candidate approach / evidence to consider	Trade-off / limitation found in research	Decision M3 must still make
Automation and quality gates improve repeatability, but a green pipeline only proves that the configured checks passed; it does not prove complete quality (Fowler, 2012).	Regressions or defects may be missed when checks are incomplete, poorly designed or disconnected from requirements and real user needs.	Consider automated build/test checks and merge or release gates together with traceable test evidence, review and validation of important workflows.	Automation is limited by the scenarios and assertions that were created. Unknown defects, usability issues and untested assumptions can remain.	What evidence must CivicConnect require before merge and before release, and which checks should be automated versus supported by human review or validation?
Secure design, secure coding and dependency/security checks complement one another; automated tools or AI recommendations cannot establish that a system is secure on their own (OWASP, 2026; Al-Qadda et al., 2026).	Design weaknesses, implementation flaws, vulnerable dependencies or context-specific misuse may remain even when automated scans report no high-severity finding.	Consider combining design review, secure coding practices, dependency/static checks and targeted security verification where justified by identified threats.	Automated tools can produce false positives/negatives and may not understand business logic, architecture context or missing requirements.	Which security checks and human review activities are appropriate for CivicConnect's actual stack, dependencies and identified threat model in M3?
Environment parity matters because uncontrolled differences between development, test/staging and production can create deployment failures even when the software works locally (Microsoft, 2023).	Configuration drift, environment-specific defects and successful tests that do not represent the release environment.	Consider controlled environment-specific configuration, repeatable deployment and a staging environment that is sufficiently representative of production for release evidence.	Perfect parity may be impractical because of cost, service limits or unavailable production data. A representative environment still cannot reproduce every production condition.	What environment structure and degree of parity are practical for CivicConnect after M2 technology/deployment decisions, and what evidence is required before progression to release?
Secrets should be separated from source code and access should be controlled, auditable and limited according to least privilege (OWASP, 2026).	Credential or configuration leakage through repositories, deployment pipelines or overly broad access.	Consider environment-specific secret/configuration mechanisms, restricted access, auditability and checks that sensitive values are not committed to source control.	Secret-management approaches add configuration and operational complexity, and incorrect permissions or handling can still expose sensitive values.	Which secrets and sensitive configuration will exist in the selected CivicConnect stack, and how should they be stored, accessed, rotated or verified during M3?
Production readiness requires recovery and observability evidence, not only successful deployment: teams must be able	Failed releases, data loss/corruption, undetected incidents and inability to	Consider rollback/recovery evidence, backup/recovery checks, and actionable logs, metrics, health information or	Rollback can be complicated by database/data changes; backups are useful only if recoverable; observability	Which CivicConnect failure scenarios require rollback or data recovery, and which operational signals are necessary to

to detect failure, understand it and recover from it (Hilton, 2024; IBM, 2025).	diagnose problems after deployment.	alerts appropriate to the architecture and operational risks.	creates noise and overhead when data is not actionable.	demonstrate release readiness without over-engineering the project?
Operational quality includes reliability, performance, scalability, cost and supportability; a system may function correctly yet still be unsuitable to operate sustainably (Cox et al., 2025).	Poor response time, capacity limits, excessive resource use, free-tier exhaustion or a system that is difficult to support after release.	Consider measurable performance/reliability evidence, realistic workload checks, resource/cost observation and a support approach proportionate to expected use.	More resilience, capacity and monitoring can increase complexity and cost. Expected workload and free-tier limits may be uncertain before real use.	Which operational quality thresholds and workload assumptions are realistic for CivicConnect, and what evidence is necessary to show that the selected architecture/platform can support them?

References

Al-Qadda, F. A., Alhumaidi, A. Y. & Saqib, N. A., 2026. Secure Programming and Secure Design in DevSecOps: A Literature-Based Conceptual Synthesis. *Network,* 6(3), pp. 240-256.

Blancaflor, E. B. et al., 2024. *Mitigating Injection and Phishing Attacks through Input Validation and Phishing Detection Techniques.* s.l., s.n., pp. 13-20.

Cox , R., Knotz , H., Lees, D., Gracio, F., Harmson, G., Andrade, J. and Dhanagopal, K. (2025). *Google cloud well-architected framework*. [online] Google Cloud Documentation. Available at: https://docs.cloud.google.com/architecture/framework/printable [Accessed 23 Sept. 2026].

Crispin, L. and Gregory, J., 2009. Agile Testing: A Practical Guide for Testers and Agile Teams. Boston: Addison-Wesley. Accessed through O'Reilly.

Dalaq, D. et al., 2025. *A Systematic Literature Review on Static Application Security Testing (SAST) Tools: Evaluation, Benchmarks, Challenges, and Future Directions.* Istanbul, Türkiye, ACM, pp. 162-168.

Ewaschuk, R. and Beyer, B. (2017). *Google - site reliability engineering*. [online] sre.google. Available at: https://sre.google/sre-book/monitoring-distributed-systems/ [Accessed 23 Sept. 2026].

Fowler, M., 2012. TestPyramid. [online] martinfowler.com. Available at: https://martinfowler.com/bliki/TestPyramid.html [Accessed 26 September 2026].

Hilton, A. (2024). *Service not allowed*. [online] Google.com. Available at: https://cloud.google.com/blog/products/gcp/reliable-releases-and-rollbacks-cre-life-lessons [Accessed 23 Sept. 2026].

Humble, J. and Farley, D., 2010. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston: Addison-Wesley. Accessed through O'Reilly.

IBM (2021). *Backup and restore*. [online] Ibm.com. Available at: https://www.ibm.com/think/topics/backup-and-restore [Accessed 23 Sept. 2026].

IBM (2025). *Three pillars of observability: Logs, metrics and traces*. [online] Ibm.com. Available at: https://www.ibm.com/think/insights/observability-pillars [Accessed 23 Sept. 2026].

Mathenge, J. (2020). *Deploying vs releasing software: What's the difference?* [online] BMC Blogs. Available at: https://www.bmc.com/blogs/software-deployment-vs-release/ [Accessed 23 Sept. 2026].

Microsoft (2023). *Environments - cloud adoption framework*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/considerations/environments [Accessed 23 Sept. 2026].

Microsoft, 2022. *Microsoft Threat Modeling Tool.* [Online]
Available at: https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool?utm_source=copilot.com
[Accessed 26 09 2026].

Mohamed, A., Auer, D., Hofer, D. & Küng, J., 2022. A systematic literature review for authorization and access control: definitions, strategies and models. *International Journal of Web Information Systems,* 18(2/3), pp. 156-180.

Mweu, B. & Ndia, J., 2025. Static Analysis Techniques for Secure Software: A Systematic Review. *Journal of Cyber Security,* 7(1), pp. 417-473.

Oracle (2026). *Managing deployment pipelines*. [online] Oracle.com. Available at: https://docs.oracle.com/en-us/iaas/Content/devops/using/deployment_pipelines.htm [Accessed 23 Sept. 2026].

OWASP, 2025a. *Input Validation Cheat Sheet.* [Online]
Available at: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
[Accessed 26 09 2026].

OWASP, 2025b. *Vulnerable Dependency Management Cheat Sheet.* [Online]
Available at:
https://cheatsheetseries.owasp.org/cheatsheets/Vulnerable_Dependency_Management_Cheat_Sheet.html
[Accessed 26 09 2026].

OWASP, 2026a. *Authentication Cheat Sheet.* [Online]
Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
[Accessed 26 09 2026].

OWASP, 2026b. *Authorization Cheat Sheet.* [Online]
Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
[Accessed 26 09 2026].

OWASP, 2026c. *Threat Modeling Cheat Sheet.* [Online]
Available at:
https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html?utm_source=copilot.com
[Accessed 26 09 2026].

OWASP, 2026d. Secrets Management Cheat Sheet. [Online] Available at: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html [Accessed 26 09 2026].

Rouland, Q., Hamid, B. & Jaskolka, J., 2021. Specification, detection, and treatment of STRIDE threats for software components: Modeling, formal methods, and tool support. *Journal of Systems Architecture,* 117(5).

Richards, M. and Ford, N., 2020. Fundamentals of Software Architecture. Sebastopol, CA: O'Reilly Media.

Satapathy, P.R. (2008). *Evaluation of Software Release Readiness Metric [0, 1] across the software development life cycle*. [online] Available at: https://www.researchgate.net/publication/229001077_Evaluation_of_Software_Release_Readiness_Metric_0_1_across_the_software_development_life_cycle [Accessed 22 Sept. 2026].

Shen, Y., Gao , X., Sun, H. & Guo, Y., 2025. Understanding vulnerabilities in software supply chains. *Empirical Software Engineering,* 30(1), p. 20.

Sommerville, I., 2015. Software Engineering. 10th ed. Boston: Pearson. Accessed through O'Reilly

Souppaya, M., Scarfone, K. & Dodson, D., 2022. *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities,* s.l.: National Institute of Standards and Technology.

Tran, A.-D., Verreydt, S., Yskout, K. & Joosen, W., 2026. A Multivocal Literature Review on the Effectiveness of Security Threat Modeling. *IEEE Transactions on Software Engineering,* 52(4), pp. 1352-1370.

Xiong, W. & Lagerström, R., 2019. Threat modeling – A systematic literature review. *Computers and Security,* 84(53-69), pp. 53-69.

AI usage register

Date	Student	Tool	Engineering Task	AI Contribution	Verification	Decision	Issues Found
26/09/ 2026	Waldo	Claude	Q3 Production-Readiness Evidence Matrix completion	Structured before/after release evidence for six identified production-readiness concerns (environment parity, secrets management, rollback/recovery, backup/data recovery, observability, monitoring/alerting). I identified all six "Production-readiness concerns" and "the Risk if ignored" for all six myself.	I reviewed the content against the assignment research sources (Microsoft, Google, IBM etc.) and confirmed it aligns with Q3 production-readiness areas. Evidence statements independently verified against submitted research content.	Accepted (partial).	Initial output contained some mistakes with regards to the Rollback and Recovery Readiness, Backup and Data Recovery, Observability: Metrics, Logs, and Traces etc. I also reworded it to sound more natural and more to my own writing style.
26/09/ 2026	Christiaan	Gemini	Q1 Quality Strategy & Risk-Based Verification.	Assisted in finding relevant sources in order to conclude and evidence. And assisted to find to make the visual for 1.5 and gave guidance to use tool 'Mermaid Live Editor'.	I reviewed each reference to confirm its relevance to the engineering task.	Accepted (partial).	I was not satisfied with some of the references initially suggested and re-prompted to find better more relevant sources. And the initial tools suggested for the visual for 1.5 was not suitable for the task.
28/09/ 2026	Marco	ChatGPT	Q2 Threat Modelling research and source review	Used ChatGPT to go through the OWASP documentation and identify only the parts relevant to the points I wanted to use in Q2. I also used it to give me a brief overview and comparison of recognised threat-modelling approaches suitable for software engineering.	I checked the OWASP material myself and verified the overview against the sources. I then researched STRIDE further using the OWASP and Microsoft material before choosing it as the approach for Q2.	Background only; AI output not used directly.	The initial overview was only used to help me understand the available approaches. I did not use it as assignment content and only used information after checking the original sources myself.
28/09/ 2026	Marco	ChatGPT	Q2 Threat-to-Control Traceability Table	After researching the four selected security concern areas and gathering the relevant source information, I provided the material to ChatGPT and asked it to help structure the threat → requirement/control objective → engineering control → verification evidence → residual risk chain into the required table format.	I checked each table entry against the sources I had researched, including OWASP and the academic sources used for authentication, authorisation/access control, input/data handling and dependency vulnerabilities.	Accepted (partial) after verification and editing.	Some output was not fully correct or was too general. I corrected the parts that were useful and rejected or replaced information that did not match the sources.
28/09/ 2026	Marco	ChatGPT	Q2 Grammar, spelling, phrasing and clarity review	Used ChatGPT throughout Q2 to check grammar, spelling and phrasing, and to help make the content more concise and clear without changing the technical meaning.	I reviewed every suggested change to make sure the original meaning, definitions and technical claims were still correct and remained supported by the sources.	Accepted (partial).	Some wording suggestions sounded too formal or did not sound like my writing style, so I reworded or rejected them. Only changes that kept the original meaning were used.

Date	Student	Tool	Engineering Task	AI Contribution	Verification	Decision	Issues Found
28/09/2026	Team BugBusters	ChatGPT	Q4 Research-to-M3 Engineering Map	After the team identified meaningful findings across quality, security and production readiness, we provided these findings and the decisions that still needed to be made to ChatGPT. ChatGPT was then used to help structure the Research-to-M3 Engineering Map and to complete the table using the information we supplied. It was also asked to review our individual Q1-Q3 research together with the project instructions and goals to help ensure that the map remained accurate and aligned with the purpose of Assignment 3.	The team reviewed the completed map against our original Q1-Q3 research, the Assignment 3 instructions, the CivicConnect project requirements and the decisions that still need to be made in Milestone 3. We verified that the table reflected the work already completed and that it did not present research options as final CivicConnect M3 decisions.	Accepted (partial) after team verification.	The AI was only used to structure and connect information that the team had already identified. Any wording or table content that did not accurately reflect our research, project context or the research-to-M3 decision boundary was corrected, reworded or rejected before use.